



Use Case 1

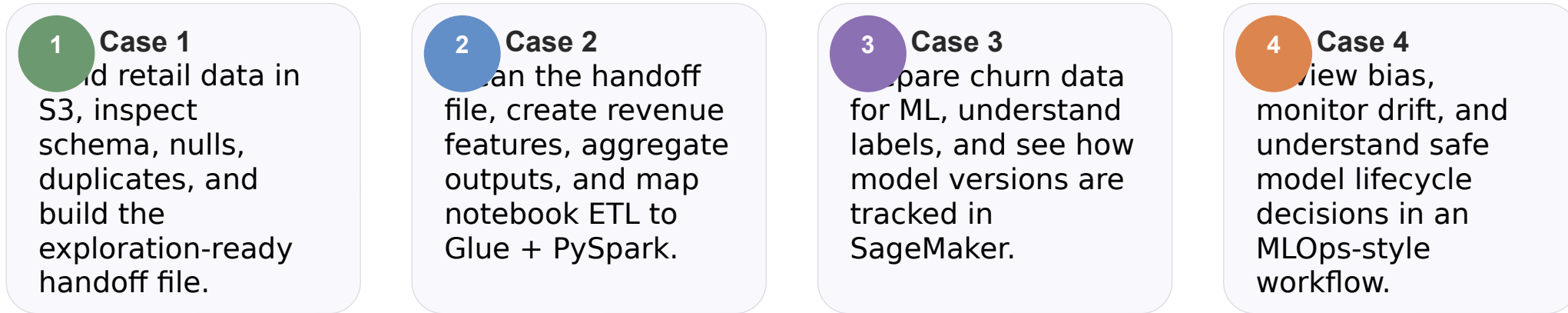
Retail Data Ingestion & Exploration

BITS learner deck • Amazon S3 + SageMaker Studio notebook

Goal: help learners understand how a raw retail CSV lands in AWS, how it is explored step by step, and how the exploration-ready file becomes the handoff for Use Case 2.

Before Use Case 1 - how all 4 use cases connect

Two sessions, four use cases, one connected AWS learning journey



Start with data understanding -> build ETL thinking -> extend to ML preparation -> finish with monitoring and responsible AI

After both sessions, learners will be able to

- explain where S3, SageMaker Studio, Glue, PySpark, and Redshift fit in one pipeline
- ingest raw CSV data from S3, inspect quality, and prepare a cloud-ready handoff file
- clean data, create revenue fields, and build ETL outputs step by step
- understand how notebook logic becomes Glue + PySpark ETL and later supports ML workflows

Before Use Case 1 - AWS services in simple words

Assume this is the first time you are hearing these names - here is the simplest meaning.

Amazon S3



Cloud storage for our files. A bucket is the main container, and objects are the files inside it. We use S3 instead of a laptop-only file system so the whole class works from one durable cloud location.

Amazon SageMaker



Managed ML and notebook platform. In this lab we use SageMaker Studio as the notebook workspace where learners run code step by step and inspect outputs in AWS.

AWS Glue



Managed service. It helps us move from notebook logic to a repeatable AWS job. Glue uses Spark underneath, so we get cloud ETL without managing servers.

PySpark / Spark



Distributed processing engine. Spark splits work across multiple workers. RDD means Resilient Distributed Dataset — useful because processing can scale and recover from failures.

Amazon Redshift

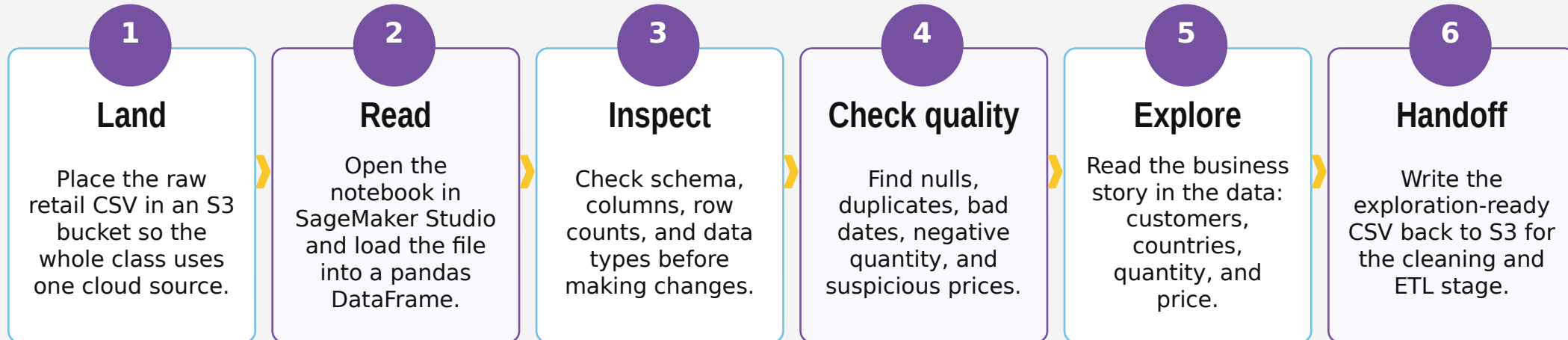


Managed data warehouse. We use it as the warehouse example so learners see where curated outputs can later land for reporting and analytics.

How to remember them: S3 stores the data, notebooks help us inspect it, Glue + Spark scale the ETL, Redshift represents the warehouse, and SageMaker supports ML work.

Use Case 1 — learner step map

Learner roadmap — follow these named stages while you code live



Use Case 1 — AWS technologies and why they fit

Simple services first, clear inner workings, and no local-path confusion during the live demo

Amazon S3



Object storage used for raw and processed CSV files.

Bucket = top container | Object = actual file

Key = path-like name
Why used here: S3 replaces a laptop-only file system because cloud services need a shared, durable source.

Sagemaker



Managed notebook environment for running the exploration notebook.

Cells combine explanation, code, output, and live discussion.

Why used here: It reduces setup risk and keeps the demo in AWS from start to finish.

boto3 + pandas



AWS-friendly read/write pattern inside the notebook.

boto3 reads from S3 directly; pandas makes inspection easy to teach.

Why used here: This removes last-minute path edits and shows a realistic S3 workflow.

Why not use only local files?

A laptop path works for one person. In an AWS lab, the class needs one shared source and cloud outputs that feed the next notebook.

What is a bucket?

Think of a bucket as the top-level storage container. Inside it you keep objects such as raw CSVs, processed CSVs, and ETL outputs.

Notebook dependency story

Imports are not random. boto3 handles AWS access, pandas handles data understanding, and io/pathlib support practical file movement.

Use Case 1 — data understanding before cleaning



Retail features learners should read before writing any transformation code

Field	What learners should notice	Why it matters in the lab
InvoiceNo	Order or invoice reference for each transaction.	Helps detect duplicates and track repeated transaction lines.
InvoiceDate	Date-time field currently delivered as text.	Tells us whether date conversion will be needed in the next notebook.
Quantity	How many items were purchased or returned.	Negative values may signal returns, not necessarily bad data.
UnitPrice	Selling price per item.	Used later to calculate revenue and catch invalid zero/negative prices.
CustomerID	Customer reference, often missing in real files.	Missing IDs affect customer-level analytics and segmentation.
Country	Where the customer order came from.	Useful for quick exploration and later country-based revenue views.

Use Case 1 — live demonstration flow



Relatable DFD: show learners how the file moves, what the notebook does, and what file is created next



What learners say here

“I can already see which columns matter and which issues will affect the next stage.”

What the code is doing

Read → inspect → count → profile → summarize. Every cell exists to reduce uncertainty about the raw data.

Why this matters

Use Case 2 becomes easier because the class already knows what must be cleaned and why.

Use Case 1 — how Notebook 1 works internally

Connect the notebook cells to the engineering purpose, dependency, and result

Setup cell

Dependencies: boto3, pandas, io, pathlib
Result: one bucket + prefix config controls the whole notebook.



Read cell

Why: load the true incoming data from S3, not a hidden cleaned copy.
Result: raw DataFrame.



Schema + quality cells

Why: understand structure, nulls, duplicates, bad dates, returns, and suspicious price values.
Result: a list of issues to explain.



Exploration + save cells

Why: link the technical checks to business meaning, then save the handoff file.
Result: retail_exploration_ready.csv in S3.

Dependencies explained simply

boto3 talks to S3. pandas gives tabular analysis. io converts S3 bytes into a file-like stream. pathlib supports local fallback for rehearsal.

Transformation of learner

~~understanding~~
The student class only sees “a CSV”.
By the end they know the structure, issues, and which columns drive later ETL.

Output to mention live

Show the processed file in S3 so learners see that notebook work created a reusable cloud artifact.

Use Case 1 — if the data changes, the interpretation changes



Use one small data change to teach engineering judgment, not only notebook execution

Scenario A — missing CustomerID

If more CustomerID values go missing, customer-level analysis becomes weaker even if revenue totals still look fine.

Scenario B — negative Quantity

A negative value may be a valid return. Do not delete it blindly; first decide whether the business treats it as a refund or an error.

Scenario C — invalid UnitPrice

If one unit price becomes zero or negative, learners should expect suspicious revenue later. This is why quality checks come before ETL.

Decision-making message for learners

“Changing one field changes the story we tell with the data. Data engineering is about making the right decision before the wrong output reaches a dashboard.”

Clean handoff to Use Case 2

End this use case by opening the processed S3 location and telling the class: “This is the file we will now clean, enrich, and aggregate.”